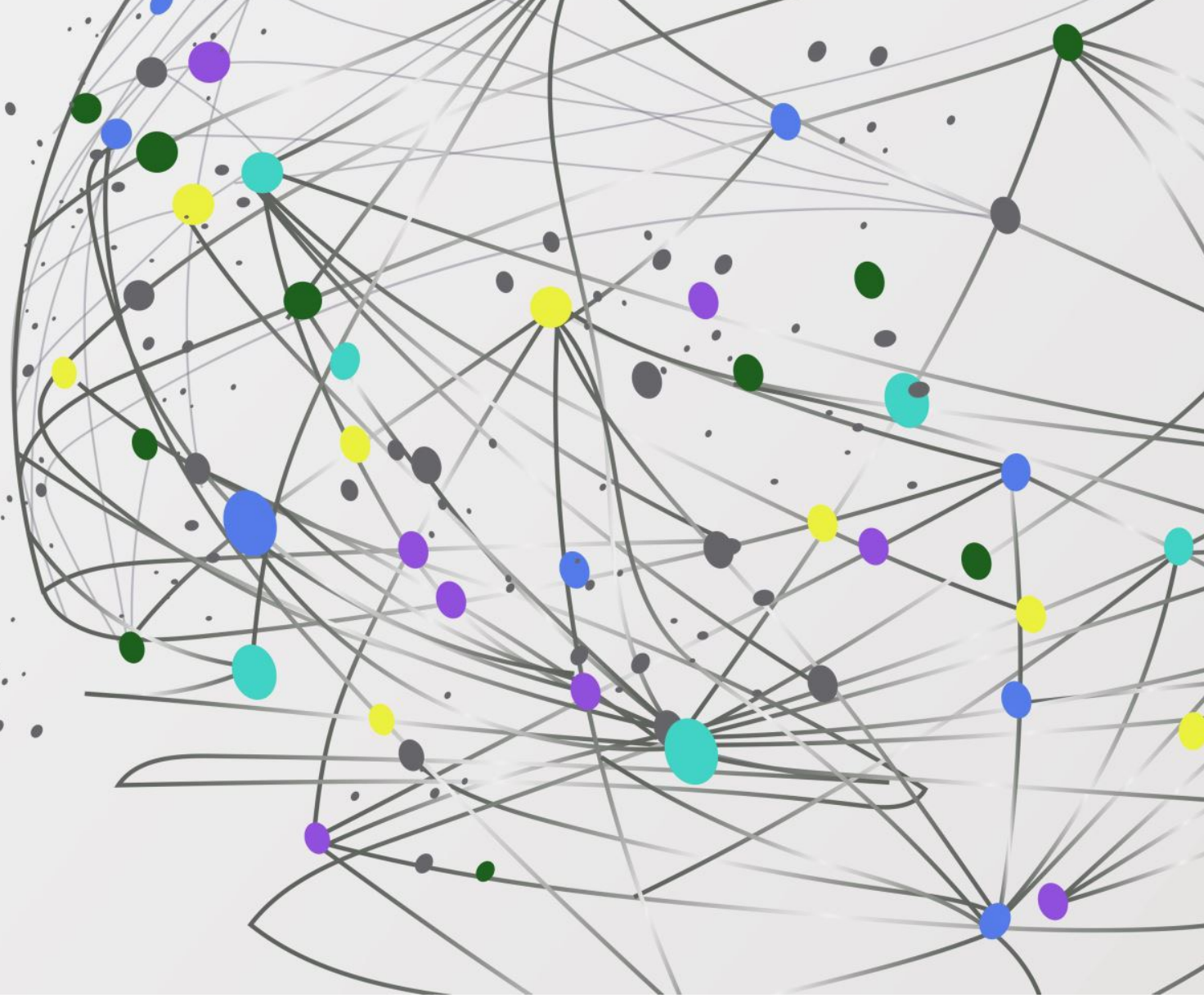
The SLEEC Framework for Normative Requirements Engineering



Radu Calinescu
Institute for Safe Autonomy &
Department of Computer Science
University of York

University of York | Institute for Safe Autonomy
University of Toronto | Schwartz Reisman Institute for Technology and Society
UK Research and Innovation

Research group @ University of York's Institute for Safe Autonomy

Theoretical foundations
Extending the theory underpinning the modelling, verification and synthesis of systems and processes with stochastic behaviour

Engineering & assurance approaches
Developing methodologies for the engineering and assurance of safety-critical AI and autonomous systems

Operationalisation of SLEEC norms
Advancing social, legal, ethical, empathetic and cultural requirements engineering for AI-enabled agents

Applied research – *Developing embodied-AI solutions for safety/mission-critical applications*

DAISY: Diagnostic AI system for robotic accident & emergency patient triage



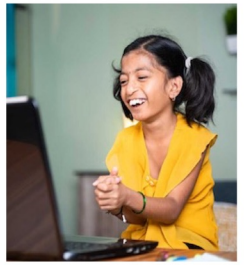
ALMI: Ambient assisted living for long-term monitoring and interaction



CSI:Cobot: Confident safety integration for collaborative robots

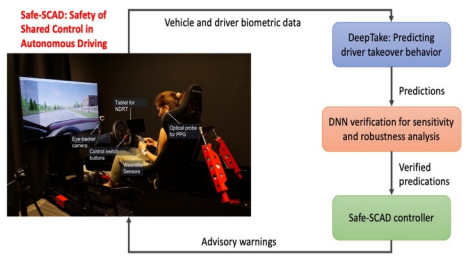


TAS-DYP: Reimagining trustworthy autonomous systems with disabled young people





SafeSCAD: Safety of shared control in autonomous driving



ASPEN: Autonomous systems for forest protection

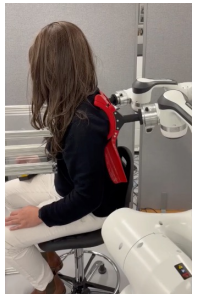


Monitoring glacier calving at Svalbard Polar Station



Breast screening support for patients with physical impairments





SLEEC Framework

Key deliverable of ongoing research programme on the operationalisation of social, legal, ethical, empathetic and cultural (SLEEC) requirements for AI-enabled agents

Operationalisation of SLEEC norms
Advancing social, legal, ethical, empathetic and cultural requirements engineering for AI-enabled agents

Applied research – *Developing embodied-AI solutions for safety/mission-critical applications*

DAISY: Diagnostic AI system for robotic accident & emergency patient triage



ALMI: Ambient assisted living for long-term monitoring and interaction



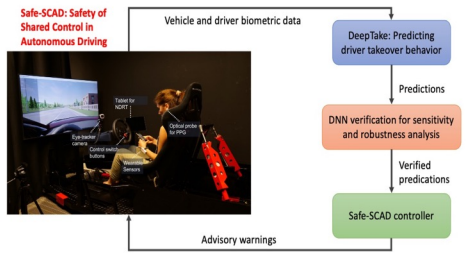
CSI:Cobot: Confident safety integration for collaborative robots



TAS-DYP: Reimagining trustworthy autonomous systems with disabled young people



SafeSCAD: Safety of shared control in autonomous driving



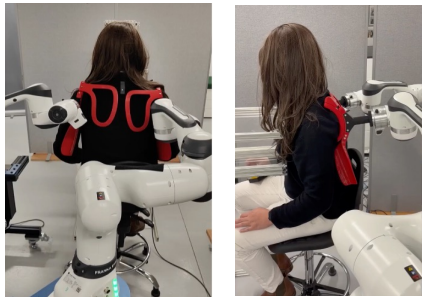
ASPEN: Autonomous systems for forest protection



Monitoring glacier calving at Svalbard Polar Station



Breast screening support for patients with physical impairments



SLEEC Framework

Key deliverable of ongoing research programme on the operationalisation of social, legal, ethical, empathetic and cultural (SLEEC) requirements for AI-enabled agents

- domain-specific language for defining SLEEC requirements
- methods for SLEEC requirement elicitation and determining agent capabilities
- algorithms for validating sets of SLEEC requirements (to identify conflicts, etc.)
- requirements engineering methodology integrating methods and algorithms into an end-to-end process
- (software) tools that automate the execution of multiple steps of the methodology
- repository of use cases illustrating application of methodology/tools

Core SLEEC-framework research team

University of York | Institute for Safe Autonomy


Radu Calinescu


T T Arvind

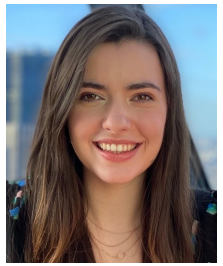

Ana Cavalcanti


Sinem Getir Yaman


A P Partis


Colin Paterson

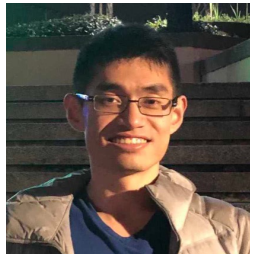

Pedro Ribeiro


Isobel Standen


Beverley Townsend

University of Toronto | Schwartz Reisman Institute for Technology and Society


Marsha Chechik


Nick Feng


Lina Marsso

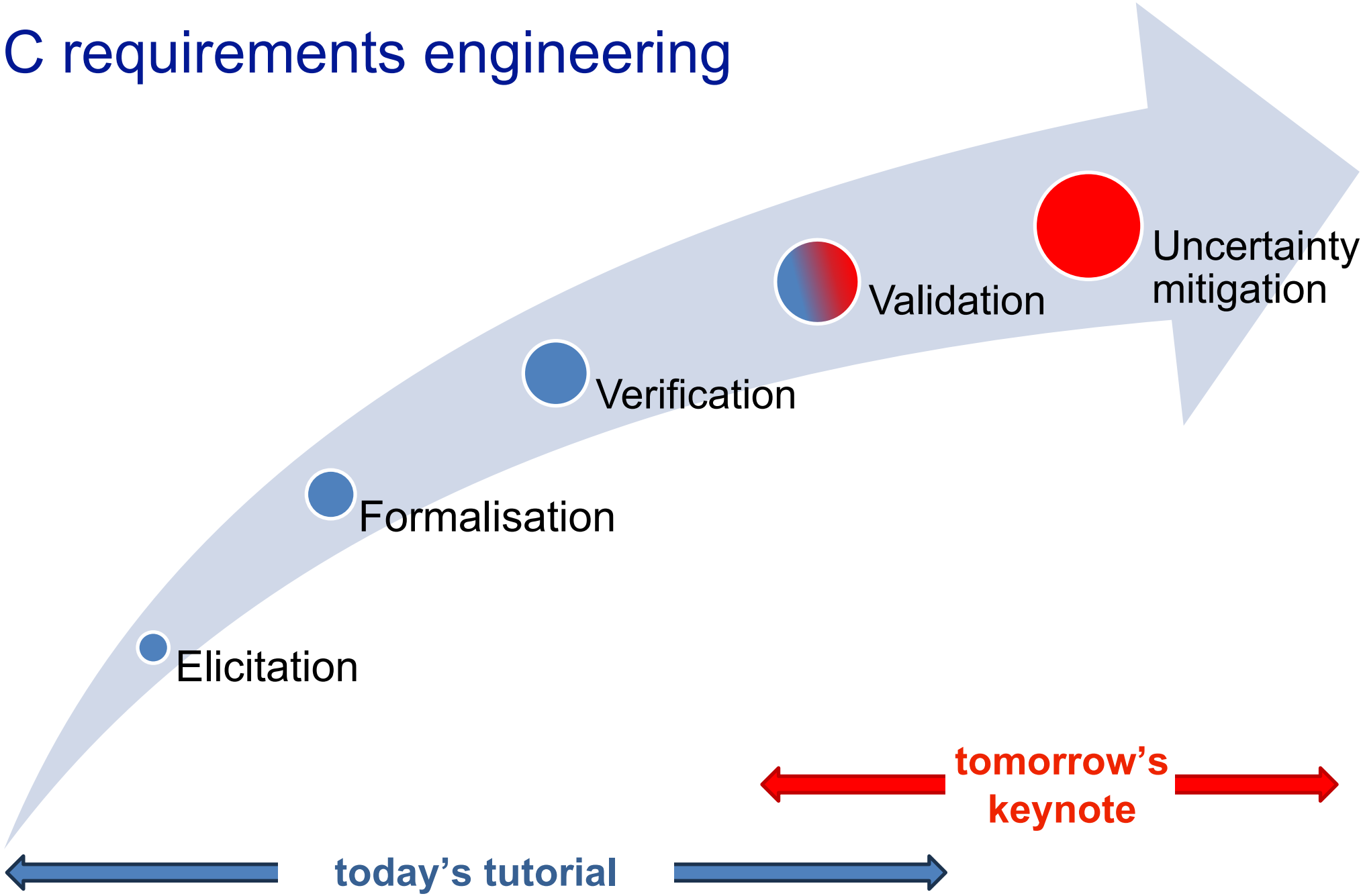
SLEEC requirements of AI agents are widely considered

Operationalising these high-level principles for a concrete, high-stakes AI agent is very challenging…



DAISY A&E patient triage system, Scarborough Hospital

SLEEC requirements engineering



Disclaimer

This talk is not about the “right” SLEEC requirements for specific (difficult) use cases…

…nor about obtaining requirement sets guaranteed to be complete.

It is not possible to produce a set of rules purporting to describe what a [person] should do in every conceivable set of circumstances.

Alan Turing

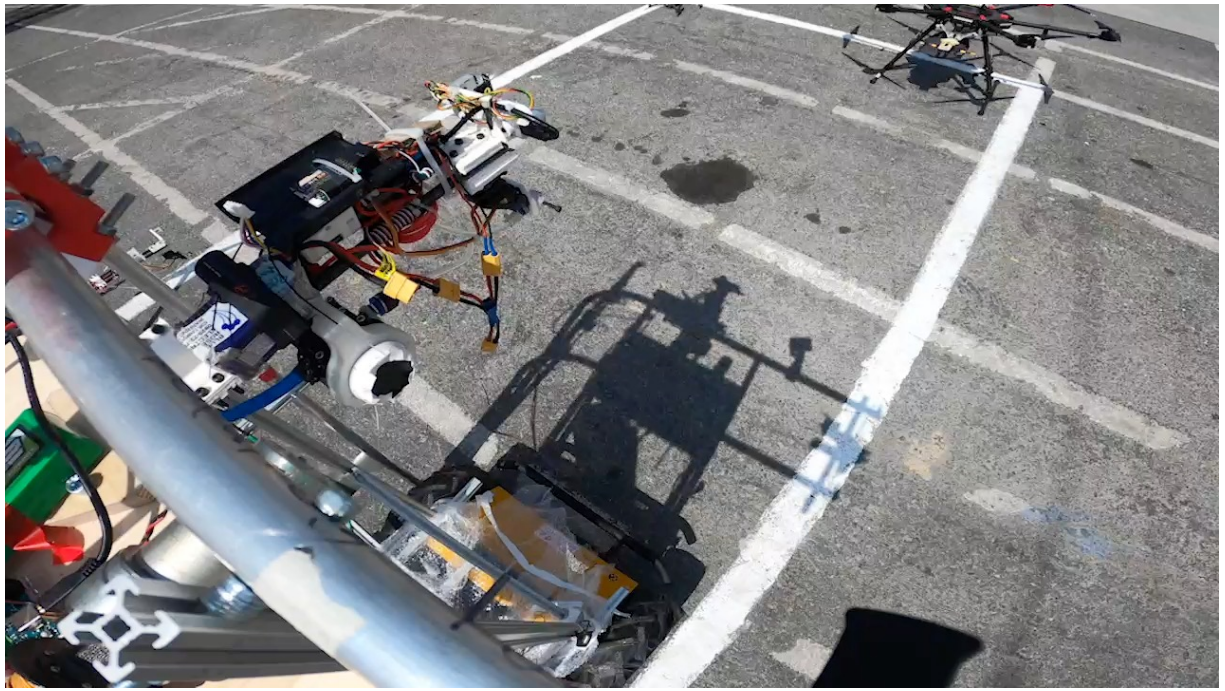


Use cases in this tutorial

Assistive-care robotics
(food preparation, dressing, fallen-user alert)



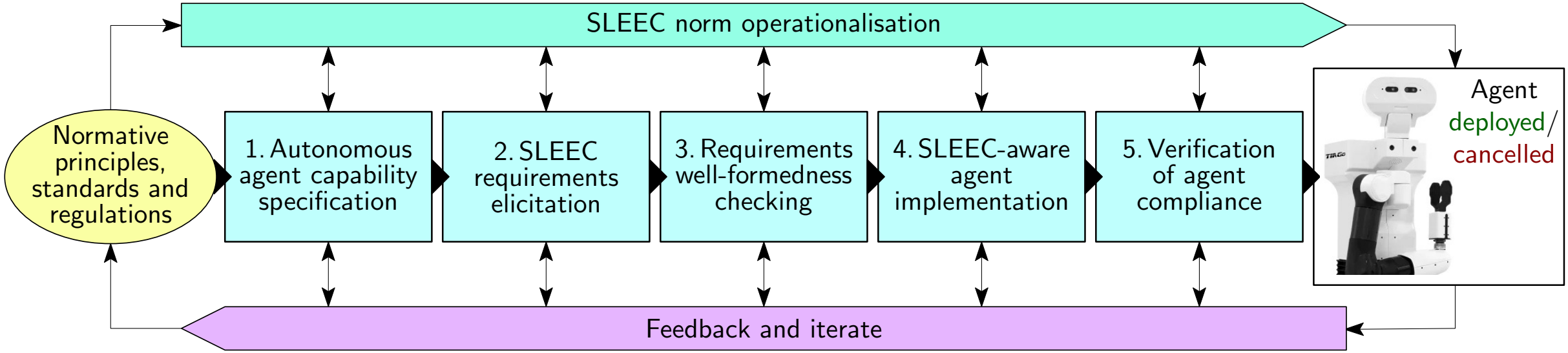
Firefighting drone
(fire detection and extinguishing, alarm)



Outline

- Motivation
- SLEEC norm operationalisation process
  - Overview
  - Stages and activities
- Takeaways

SLEEC norm operationalisation process



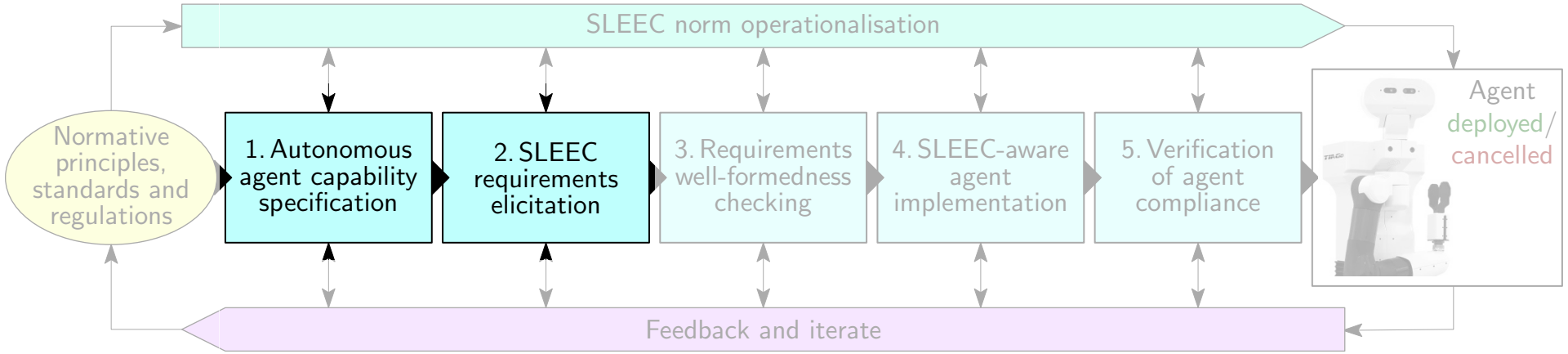
Purpose is to ensure that

Deployed autonomous agents uphold human norms & values (as understood by their stakeholders)

– OR –

Development/deployment of agents unable to do so is abandoned

SLEEC norm operationalisation process



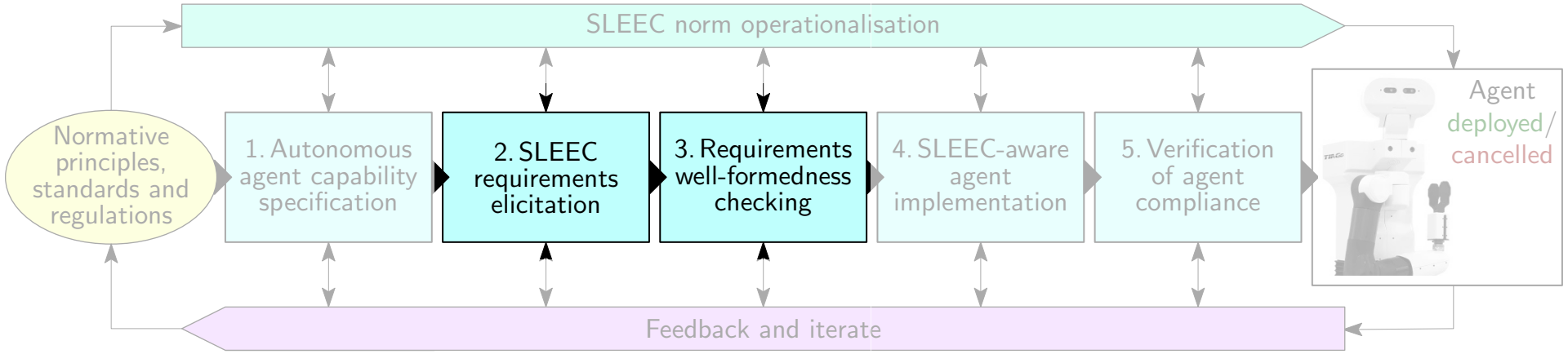
Stage 1←→2 iteration scenarios

- stakeholders demand SLEEC behaviour unsupported by planned capabilities
- planned capabilities need to be dropped due to unacceptable normative implications
- feasible capabilities require lowering SLEEC-alignment expectations

Stage 1←→2 “cancellation” scenarios

- SLEEC alignment requires capabilities that are technically unfeasible or financially prohibitive

SLEEC norm operationalisation process



Stage 2←→3 iteration scenarios

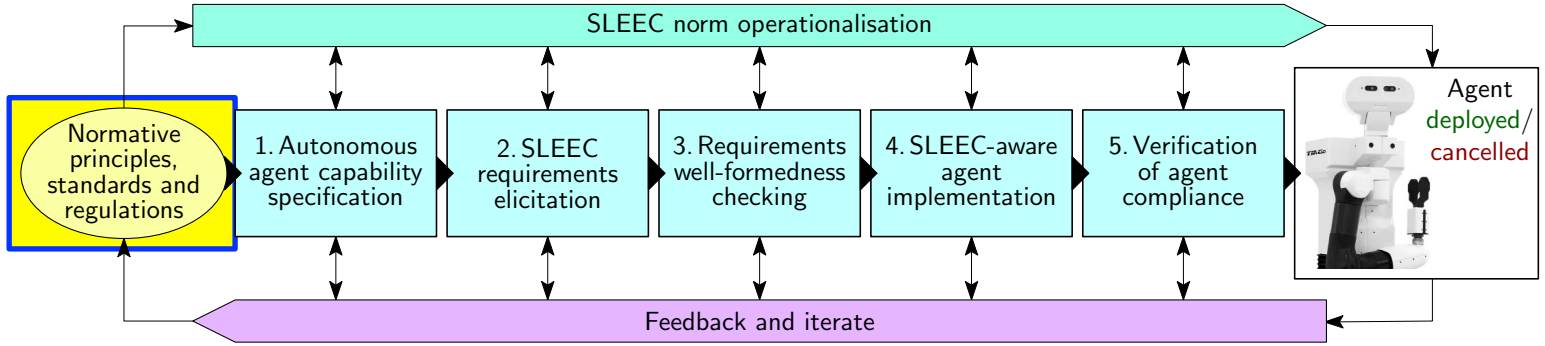
- conflicting requirements from different types of stakeholders (lawyers, ethicists, operators, end users)
- redundant requirements

Stage 2←→3 “cancellation” scenarios

- compromise to eliminate requirement conflicts cannot be achieved

Outline

- Motivation
- SLEEC norm operationalisation process
  - Overview
  - Stages and activities



- Takeaways

Process input: normative principles, standards and regulations

Normative frameworks

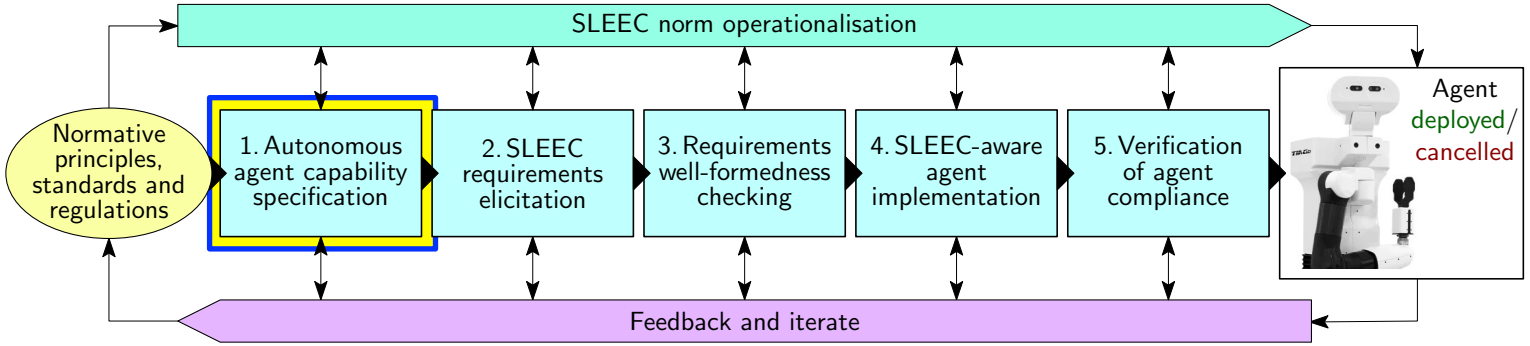
- for general-purpose autonomous/AI systems vs. domain-specific contexts (e.g., healthcare, automotive)
- international vs. regional (e.g., EU) and national level
- ethical theories, codes of conduct
- religious doctrines, customs, social conventions, etc.

Dual relationship

- process enabled by existing normative frameworks
- process application to concrete agents can identify gaps, loopholes, and drawbacks to be addressed by future normative documents

Outline

- Motivation
- SLEEC norm operationalisation process
  - Overview
  - Stages and activities



- Takeaways

Stage 1: Autonomous agent capability specification

High-level functional capabilities that determine how the agent can perceive or affect its environment

- arising from use of sensors/actuators, access to communication channels, third-party data access, etc.

Two classes of SLEEC-relevant capabilities

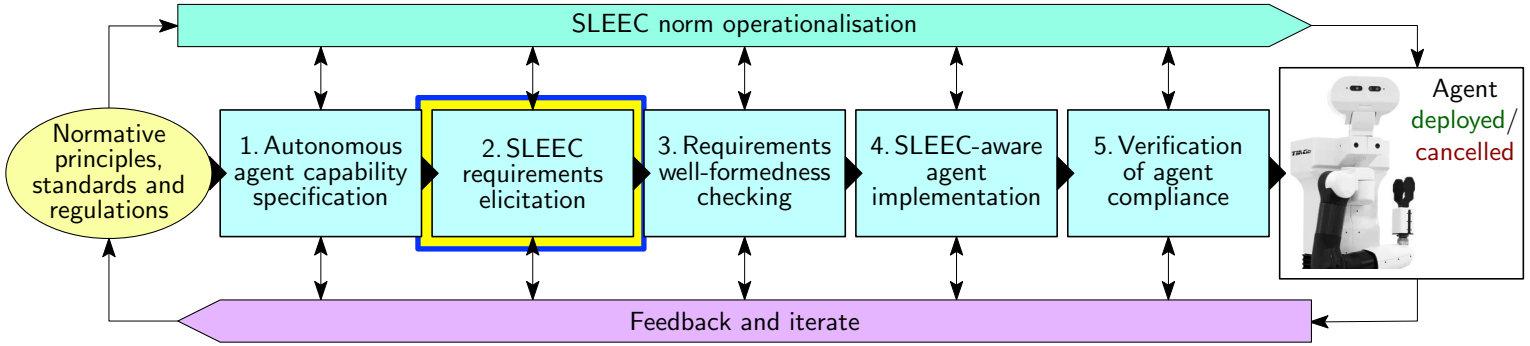
- capabilities that determine which SLEEC norms are relevant for the agent
  - e.g., robot can record video → privacy concerns
- capabilities needed to satisfy SLEEC norms
  - e.g., robot can detect patient state → robot should be able to call for assistance is patient is unwell

Stage 1: Autonomous agent capability specification

	Assistive-care robot (food preparation, dressing, fallen-user alert)	*Firefighting drone* (fire detection and extinguishing, alarm)
capabilities that determine which SLEEC norms are relevant for the agent	robot can dress the user → privacy concerns over user being exposed when underdressed	drone can record footage → privacy concerns over video recording of nearby people
capabilities needed to satisfy SLEEC norms	robot should be able to close and open the curtain	drone should be able to detect a person present in its vicinity

Outline

- Motivation
- SLEEC norm operationalisation process
  - Overview
  - Stages and activities



- Takeaways

Stage 2: SLEEC requirements elicitation

Approach to move from high-level SLEEC normative principles to a set of concrete SLEEC rules

1. Identification of **normative principles** that must inform the agent design given its capabilities and operational context.

concepts such as ‘dignity’, ‘human autonomy’, ‘benevolence’ and ‘accountability’

high-level of abstraction

guide the conduct of moral agents generally

apply across a wide range of domains

2. Derivation of **normative-principle proxies** and **proxy mappings** to agent capabilities

e.g., obtaining **assent can act as a proxy for human autonomy**, and can be mapped to a UserAssents robot capability

3. Definition of **SLEEC rules** specified in a domain-specific language (DSL)

concrete operating instructions

building on principles

narrower in scope

context dependent

intended to shape the actions, conduct and choices of the agent

Who writes the rules?

Rules co-defined by **stakeholders**

A dialogical and participatory **process of collaborative engagement** involves **deliberative and meaningful consultation** with stakeholders

- users
- domain experts (such as carers and health practitioners)
- user advocacy groups
- developers and designers
- ethicists, philosophers, and lawyers
- community leaders
- members of the public



SLEEC ruleset for the assistive care robot (partial)

def_start
  event CurtainOpenRequest
  event CurtainsOpened
  event RefuseRequest
  measure userUnderDressed: **Boolean**
  …
def_end

events and *measures* must correspond to capabilities of the autonomous agent

definition block

rule_start

BASE/STEM RULE

rule block

Rule1 **when** CurtainOpenRequest **then** CurtainsOpened **within** 60 **seconds**

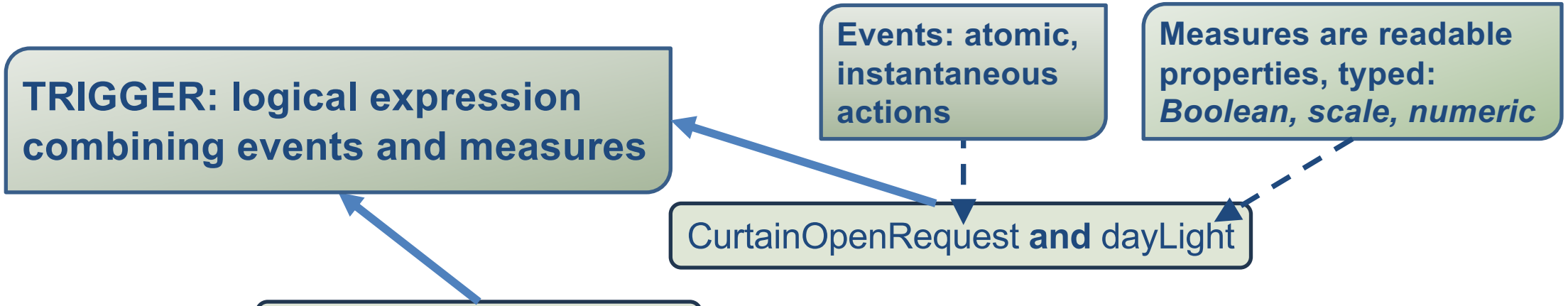
unless userUnderDressed **then** RefuseRequest **within** 30 **seconds**

RULE ID
…
rule_end

DEFEATER

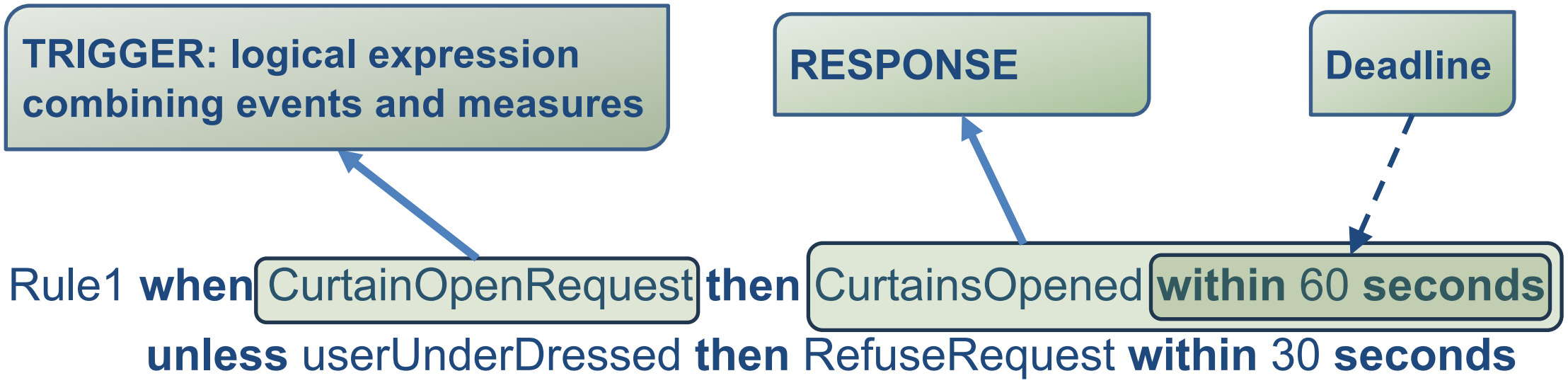
defeaters define circumstances under which a rule (and, if applicable, other defeaters) don't apply

A SLEEC rule for the assistive-care robot



Rule1 **when** CurtainOpenRequest **then** CurtainsOpened **within** 60 **seconds**
unless userUnderDressed **then** RefuseRequest **within** 30 **seconds**

A SLEEC rule for the assistive-care robot



SLEEC ruleset for the firefighter drone



Capabilities ⟷ *Rules*

```
def_start
    event BatteryCritical
    event AlarmSounding
    measure personNearby: boolean
    event CameraStart
    event VideoRecord
    event EquipmentCheck
    event AreasCheck
    constant MAX_FOOTAGE_TIME=4
def_end
```

```
rule_start
Rule1 when BatteryCritical then AlarmSounding within 40 seconds

Rule2 when CameraStart then VideoRecord
                        within MAX_FOOTAGE_TIME seconds
                        otherwise EquipmentCheck within 2 hours
                    unless personNearby
rule_end
```

The keyword 'otherwise' specifies an alternative response for scenarios when the deadline cannot be achieved (timeout).

SLEEC ruleset for the assistive-care robot



Capabilities ⟷ *Rules*

```
def_start
   event LunchPeriodStarted
   event AdviseUserToConsiderEating
   event UserFallen
   event SupportCalled
   measure prayingTime: boolean
   measure lastMealTime: numeric
   measure personAssent: boolean
   measure stressLevel: scale(light,moderate,strong)
   constant MAX_TIME=8
def_end
```

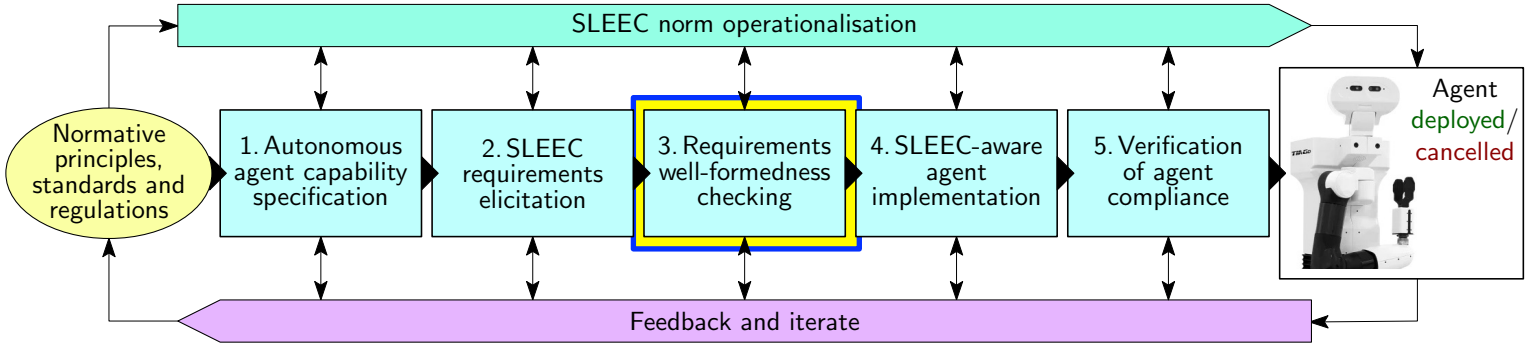
```
rule_start
  // Cultural: respect for religion & culture
  Rule1 when LunchPeriodStarted and personStressLevel>light then
          AdviseUserToConsiderEating within 2 minutes
          otherwise AdviseUserToConsiderEating within 5 minutes
          unless prayingTime
          unless lastMealTime > MAX_TIME hours then
                  AdviseUserToConsiderEating within 1 minutes

  // Ethical-Legal: privacy, emergency
  Rule2 when UserFallen then SupportCalled
          unless not personAssent

  …
```

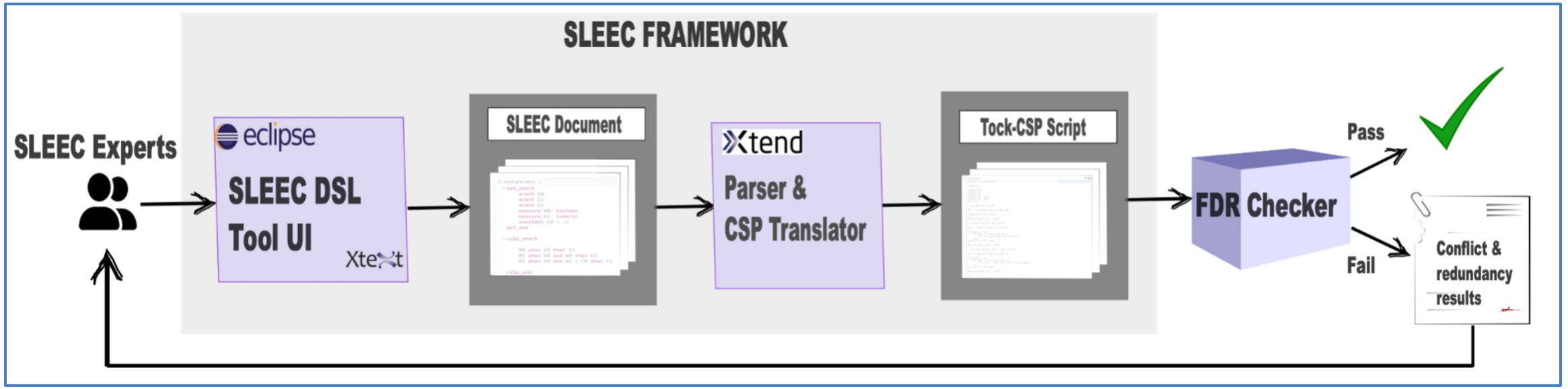
Outline

- Motivation
- SLEEC norm operationalisation process
  - Overview
  - Stages and activities



- Takeaways

Stage 3: Requirements well-formedness checking



https://github.com/UoY-RoboStar/SLEEC-TK

SLEEC semantics: tock-CSP

- Processes define patterns of interaction.
- Atomic and instantaneous events: not SLEEC events (only)
- (Non)determinism, deadlock, livelock, and termination.
- Based on CSP (communicating sequential processes) process algebra
- A special CSP event called *tock*
- For now: 1s

SLEEC semantics: overview

Rule2a **when** UserFallen **then** SupportCalled

- SLEEC events and measures become CSP events.
- Measures are always available: read with deadline 0
- Rules are defined by processes.

$$Rule2a = UserFallen \rightarrow SupportCalled \rightarrow Rule2a \;\Box\; SupportCalled \rightarrow Rule2a$$

SLEEC semantics: overview

$$Rule2a = UserFallen \rightarrow SupportCalled \rightarrow Rule2a \;\square\; SupportCalled \rightarrow Rule2a$$

$$Rule2a = Trigger2a \;;\; Monitoring2a \;;\; Rule2a$$

$$Trigger2a = UserFallen \rightarrow SKIP \;\square\; SupportCalled \rightarrow Trigger2a$$

$$Monitoring2a = SupportCalled \rightarrow SKIP$$

$$RuleProcess = TriggerProcess; MonitoringProcess; RuleProcess$$

SLEEC semantics: overview

Rule23a **when** UserFallen **then** SupportCalled **within** 1 **minutes**
unless not assentToSupportCalls **then** OpenCurtain
unless emergency

```
Monitoring23a = let
   Monitoring 1 = 60◪ (SupportCalled → SKIP)
   Monitoring2 = OpenCurtain → SKIP
   Monitoring3 = SKIP
within
   0 ◪ (emergency?ve →
         if  ve == true then Monitoring3
         else 0 ◪ (assentToSupportCalls?va →
                  if  va == true then Monitoring2  else Monitoring1) )
```

Checking Conflicting SLEEC Rules: Example 1

RuleA **when** BatteryCritical **and** temperature < 25
then GoHome **within** 1 **seconds**

RuleB **when** AlarmSounding
then not GoHome **within** 5 **seconds**



No fire risk judging by the low temperature, and the battery reaches a critical level.

AlarmSounding indicates the likely presence of a nearby fire

Question:

Can you spot the conflict between these rules?

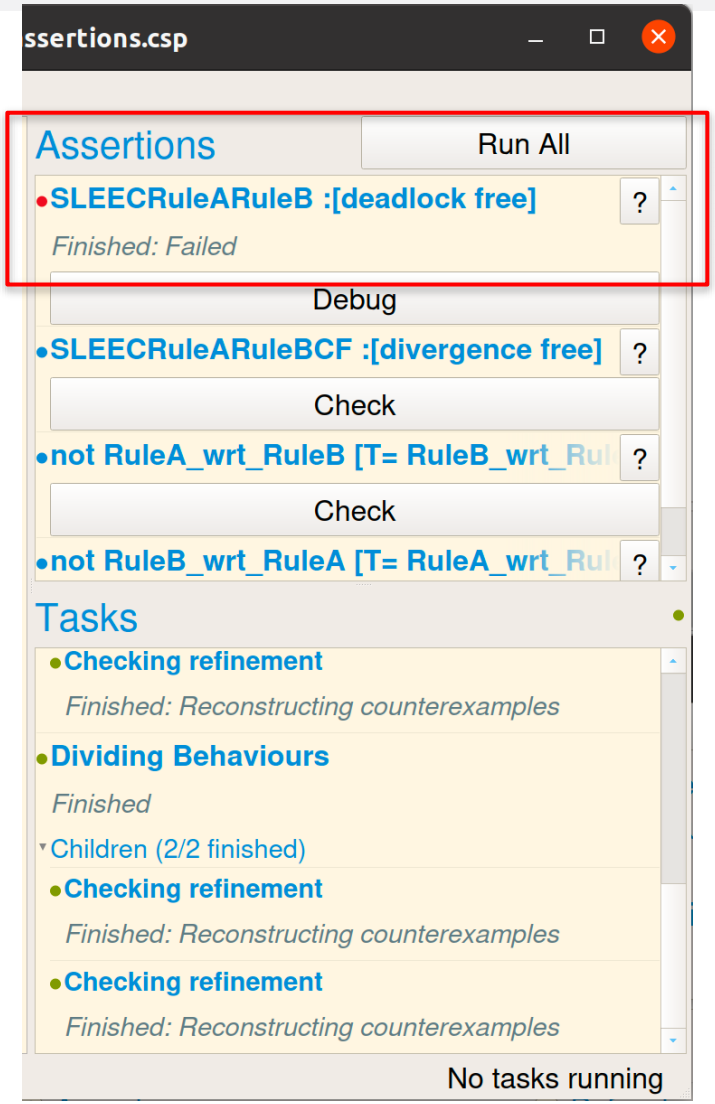
Checking Conflicting SLEEC Rules: Example 1

TOOL DEMO

RuleA **when** BatteryCritical **and** temperature < 25
then GoHome **within** 1 **seconds**

RuleB **when** AlarmSounding
then not GoHome **within** 5 **seconds**





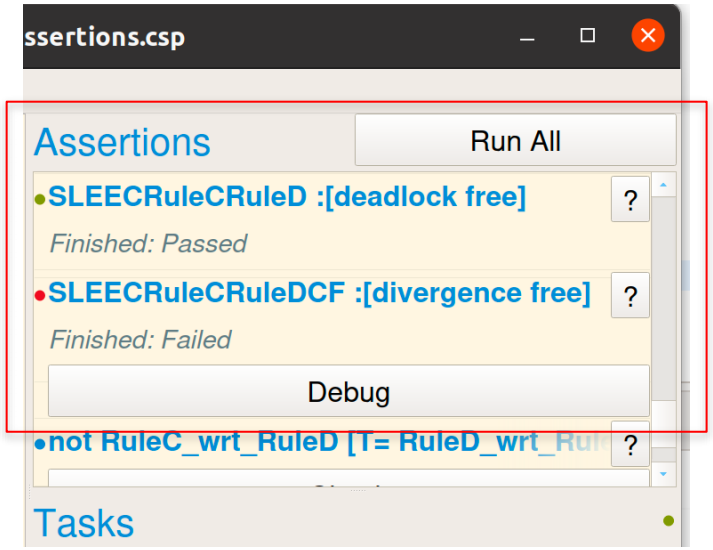
Checking Conflicting SLEEC Rules: Example 2



TOOL DEMO

RuleC **when** BatteryCritical **then** CameraStart
unless personNearby **then** GoHome
unless temperature>35 **then** SoundAlarm

RuleD **when** BatteryCritical **then** CameraStart
unless temperature>35 **then** SoundAlarm
unless personNearby **then** GoHome



No *untimed* deadlock identified. However, when (i) the battery level is critical, (ii) there is a person nearby, and (iii) the temperature is above 35 (degrees):

- RuleC demands that the drone sounds the alarm (and prevents it from going home)
- RuleD demands the drone to go home, and prevents it from sounding the alarm

This is identified as a different type of conflict.

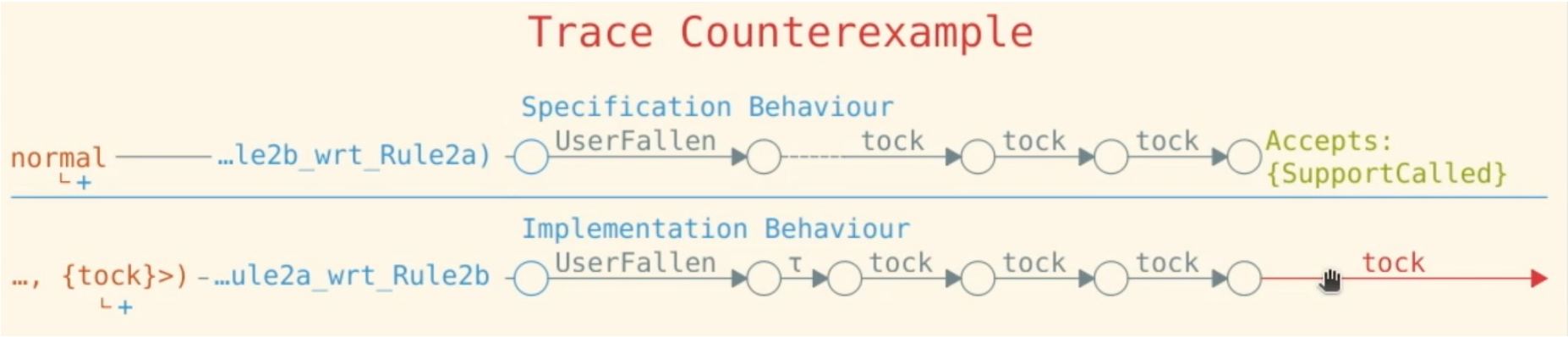
Identifying Redundant SLEEC Rules: Example 3



TOOL DEMO

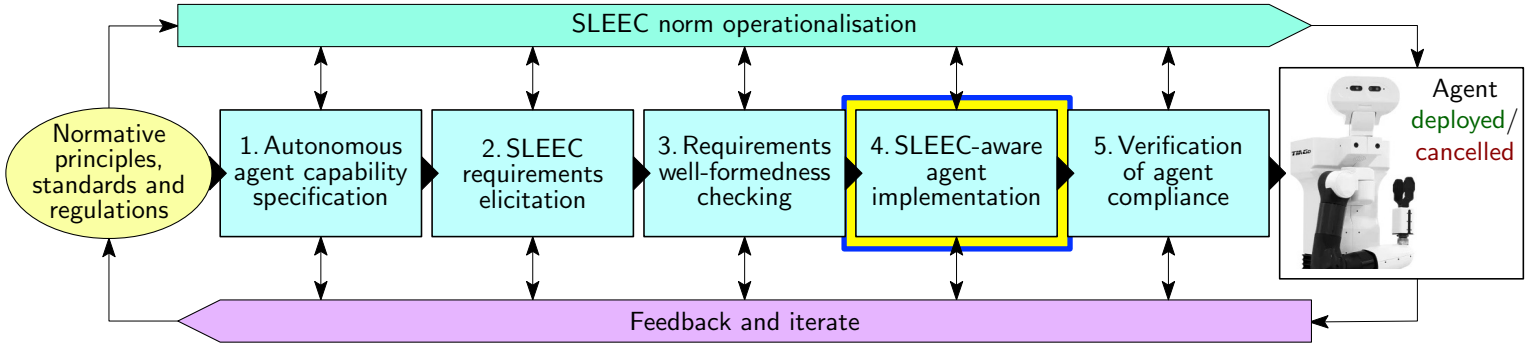
```
1 def_start
2     event UserFallen
3     event SupportCalled
4 def_end
5
6 rule_start
7     Rule2a when UserFallen then SupportCalled
8
9     Rule2b when UserFallen then SupportCalled within 3 seconds
10 rule_end
```

Rule2a is *weaker*, as it does not have a deadline, and can be removed from the requirements set.



Outline

- Motivation
- SLEEC norm operationalisation process
  - Overview
  - Stages and activities

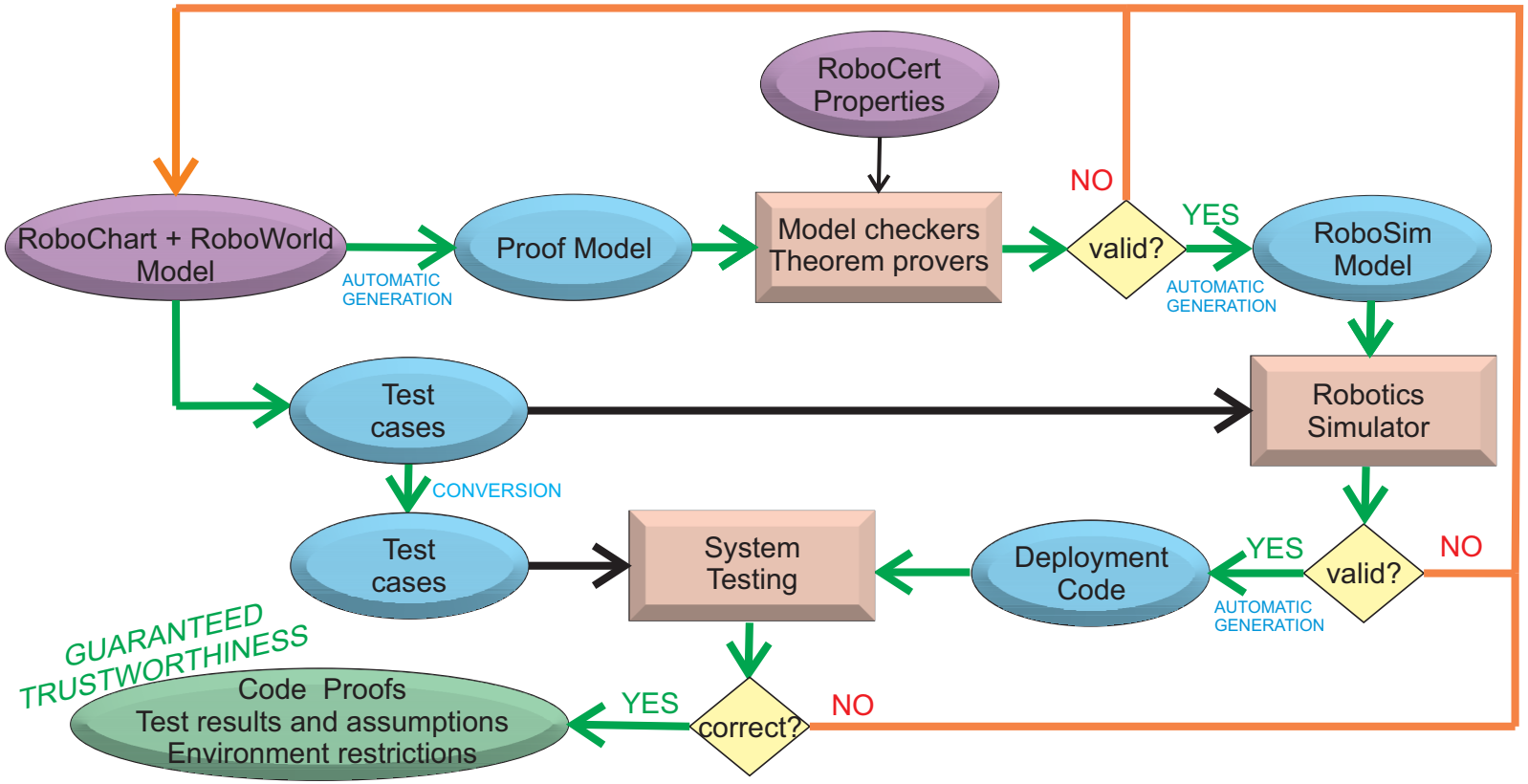


- Takeaways

Stage 4: SLEEC-aware autonomous agent implementation

Use RoboStar software engineering for robotics toolset to develop agent RoboChart model

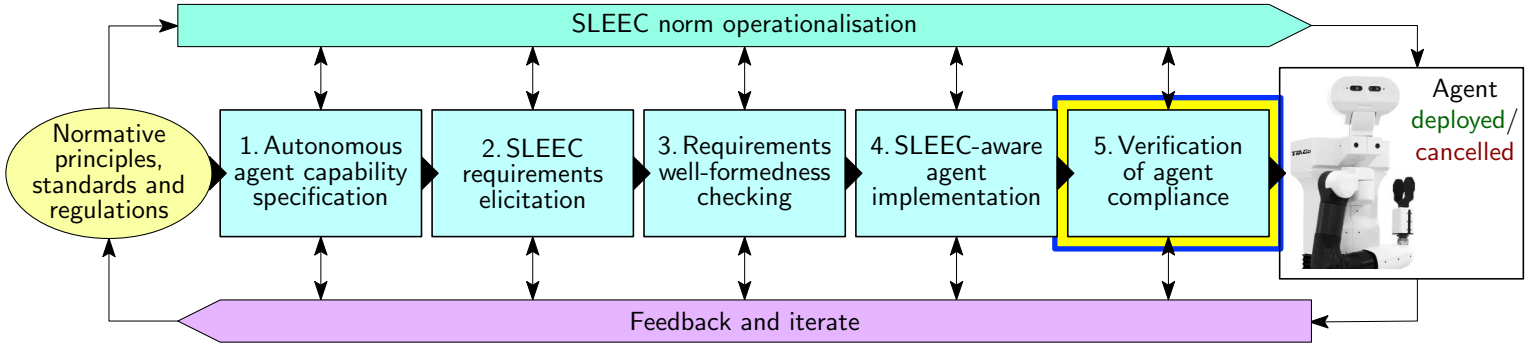
- can be used to synthesise agent code and tock CSP semantics (same semantics as for the SLEEC rules)



ROBOSTAR

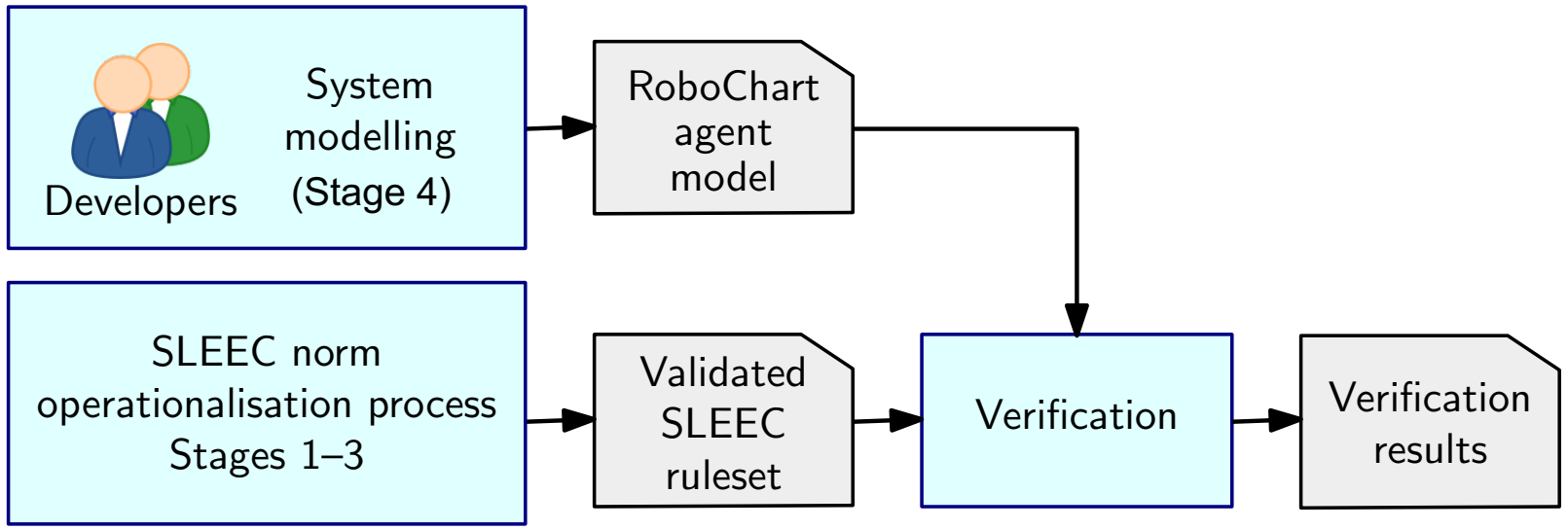
Outline

- Motivation
- SLEEC norm operationalisation process
  - Overview
  - Stages and activities

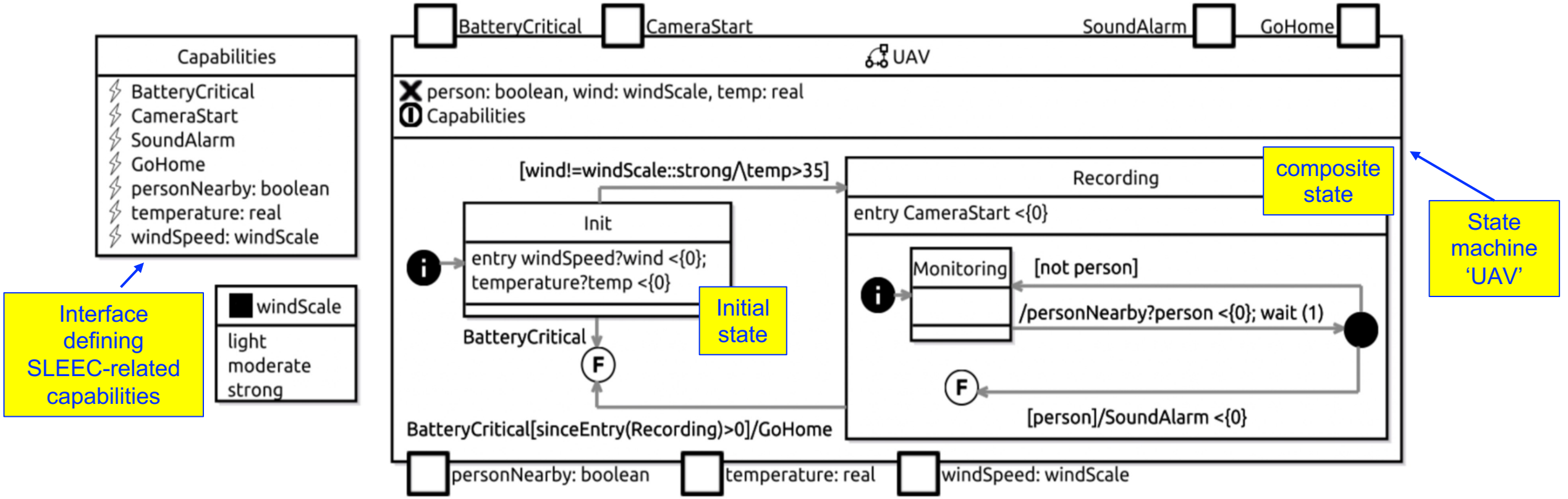


- Takeaways

Stage 5: Verification of agent compliance with SLEEC ruleset



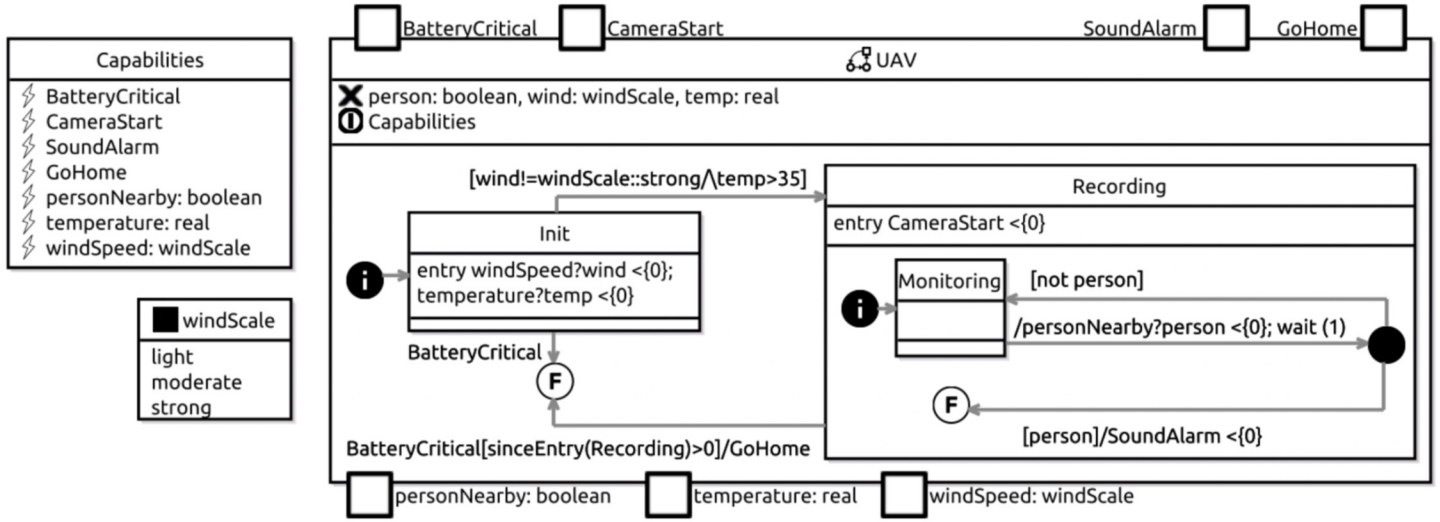
System modelling in RoboChart: firefighter drone



Miyazawa A, Ribeiro P, Li W, Cavalcanti A, Timmis J, Woodcock J. RoboChart: modelling and verification of the functional behaviour of robotic applications. *Software & Systems Modeling* **18**:3097-149, 2019.

Verifying firefighter drone compliance with SLEEC rules

TOOL DEMO



rule_start

Rule2 **when** CameraStart **and** personNearby **then** SoundAlarm **within** 2 **seconds**

Rule3 **when** SoundAlarm **then not** GoHome **within** 5 **minutes**

rule_end

windSpeed.light, temperature.36, CameraStart, personNearby.true, tock, SoundAlarm, BatteryCritical, GoHome

COUNTER EXAMPLE TRACE FOR R3

Outline

- Motivation
- SLEEC norm operationalisation process
  - Overview
  - Stages and activities
- Takeaways

Takeaways

Trustworthiness of AI-enabled agents is a sociotechnical endeavour

- Technical engineering must be accompanied by alignment with human norms & values
- Overlooked in practice due to challenges around interdisciplinary, stakeholder participation, unclear financial incentives

The SLEEC framework mitigates many challenges

- Structured normative requirements engineering process
- Formal specification language
- Tool-supported validation and verification techniques

Join us in tackling the remaining challenges

- Dynamic resolution of normative ambiguity and value conflicts
- Runtime adaptation to evolving normative requirements
- Enhancement of stakeholder normative competence
- Extension to foundation-model AI agents

Resources

FM 2026 tutorial paper



Use case repository



Theoretical foundation





Tooling



Extensions







